

SkyRoute Planner

Manual Técnico del Sistema

Universidad de Caldas

Ingeniería de Sistemas y Computación

Estructuras de Datos — Proyecto de Grafos

Versión 1.0 · Junio 2026

Autores: Javier Mosquera, Nicolas Ramirez, Simon Isaza

Tabla de Contenidos

1. Descripción General del Sistema
2. Arquitectura del Sistema
 1. Capas de la aplicación
 2. Comunicación entre capas
 3. Sistema de eventos del frontend
3. Backend
 1. Punto de entrada — app.py
 2. Modelos de datos
 3. Algoritmos implementados
 4. Capa de servicios
 5. Controladores y API REST
 6. Formato del archivo de datos JSON
 7. Utilidades
4. Frontend
 1. Estructura de archivos
 2. Vistas HTML
 3. Flujo de la interfaz de usuario
 4. Servicios JavaScript
 5. Page Managers
 6. Motor de renderizado — GraphRenderer
 7. Sistema de eventos personalizados
 8. Estilos CSS
5. Requisitos del Sistema
6. Instalación y Ejecución

1. Descripción General del Sistema

SkyRoute Planner es una aplicación web de escritorio para planificación de rutas aéreas basada en teoría de grafos. Permite cargar una red de aeropuertos y rutas desde un archivo JSON, visualizarla interactivamente sobre un canvas, y resolver distintos problemas de optimización de viajes mediante algoritmos clásicos de grafos.

El sistema cubre cinco módulos funcionales principales:

- 1. **Carga y visualización de la red aérea** — grafo interactivo con simulación de fuerzas.
- 2. **Planificación de rutas óptimas** — rutas mínimas por distancia, tiempo, costo o combinaciones; también itinerarios automáticos con restricciones de presupuesto y tiempo.
- 3. **Viaje dinámico paso a paso** — simulación de viaje con gestión de presupuesto, alojamiento, alimentación, actividades y trabajos.
- 4. **Gestión de interrupciones** — bloqueo y desbloqueo de rutas, detección de vuelo en tránsito y re-enrutamiento automático.
- 5. **Reporte final** — resumen completo de destinos, vuelos, actividades, trabajos y totales financieros.

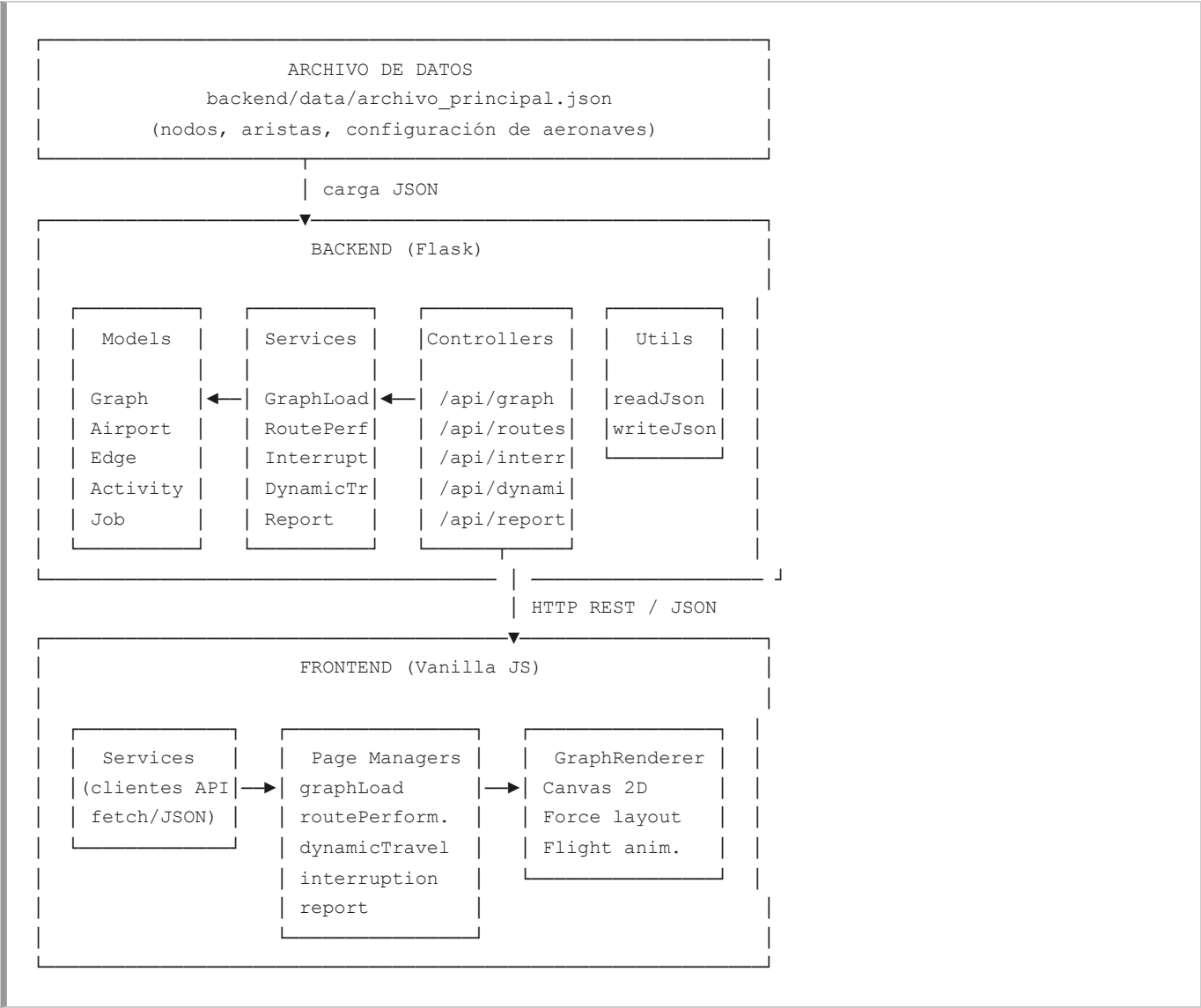
Stack tecnológico

Capa	Tecnología	Versión usada
Servidor web / API	Python + Flask + Flask-CORS	Python 3.8+, Flask 2.0+
Lógica de grafos	Python puro (sin librerías externas de grafos)	—
Interfaz de usuario	HTML5 + CSS3 + JavaScript ES6 (módulos nativos)	Sin bundler ni transpilador
Visualización	Canvas API (2D context)	Nativo en navegadores modernos
Persistencia	Ninguna — grafo completamente en memoria	—

El backend no usa base de datos. El grafo se construye en memoria al iniciar la sesión cargando el archivo JSON; se pierde al reiniciar el servidor o al hacer reset desde la interfaz.

2. Arquitectura del Sistema

2.1 Capas de la aplicación



2.2 Comunicación entre capas

El frontend se comunica con el backend exclusivamente mediante peticiones HTTP a la API REST expuesta por Flask. Todas las respuestas son JSON con el siguiente esquema:

```
// Respuesta de éxito
{
  "status": "success",
  "message": "...", // opcional
  "data": { ... }
}

// Respuesta de error
{
  "status": "error",
  "message": "Descripción del error"
}
```

Flask sirve los archivos estáticos del frontend directamente; la carpeta `frontend/` está montada como carpeta estática, por lo que `/css/app.css` apunta a `frontend/css/app.css`. No se requiere ningún servidor adicional.

2.3 Sistema de eventos del frontend

Los Page Managers se comunican entre sí usando eventos personalizados del DOM (`CustomEvent`) disparados sobre `document`. Esto desacopla los módulos evitando referencias directas entre ellos.

Evento	Disparado por	Escuchado por	Propósito
<code>skyroute:networkLoaded</code>	graphLoadManager	routePerf., dynamicTravel, interruption	Red cargada — poblar selectores de aeropuertos
<code>skyroute:networkReset</code>	graphLoadManager	Todos los managers	Reinicio completo del sistema
<code>skyroute:nodeSelected</code>	graphLoadManager	—	Nodo clickeado en canvas
<code>skyroute:edgeSelected</code>	graphLoadManager	—	Arista clickeada en canvas
<code>skyroute:deselect</code>	graphLoadManager	—	Clic en espacio vacío
<code>skyroute:journeyEnded</code>	dynamicTravelManager	reportManager	Viaje terminado — generar reporte
<code>skyroute:journeySnapshot</code>	dynamicTravelManager	reportManager	Snapshot de estado durante el viaje
<code>skyroute:flightInterrupted</code>	interruptionManager	dynamicTravelManager	Vuelo en tránsito bloqueado — revertir estado
<code>skyroute:routeRecalculated</code>	interruptionManager	dynamicTravelManager	Nueva ruta calculada
<code>skyroute:routeBlocked</code>	interruptionManager	dynamicTravelManager	Ruta bloqueada — refrescar opciones de vuelo

3. Backend

3.1 Punto de entrada — backend/app.py

Es el archivo que lanza el servidor Flask. Define la función de fábrica `create_app()` que configura rutas estáticas, habilita CORS y registra los cinco blueprints de la API.

```
app = Flask(
    __name__,
    template_folder="frontend/views",    # plantillas HTML
    static_folder="frontend",            # archivos estáticos
    static_url_path=""                   # servidos desde la raíz
)
CORS(app)    # permite fetch() desde cualquier origen (desarrollo)

app.register_blueprint(graph_load_bp,      url_prefix="/api/graph")
app.register_blueprint(route_performance_bp, url_prefix="/api/routes")
app.register_blueprint(report_bp,          url_prefix="/api/report")
app.register_blueprint(interruption_bp,    url_prefix="/api/interruptions")
app.register_blueprint(dynamic_travel_bp,   url_prefix="/api/dynamic")

# Ejecución:  python backend/app.py
#              http://localhost:5000
```

El servidor escucha en `0.0.0.0:5000` con modo debug activo. En producción se debe cambiar a `debug=False` y usar un servidor WSGI como Gunicorn.

3.2 Modelos de datos

Los modelos son clases Python que representan las entidades del dominio. No usan ORM ni base de datos; el grafo vive en memoria mientras el servidor esté activo.

Vertex — backend/models/vertex.py

Clase base para todos los nodos del grafo.

Atributo / Método	Tipo	Descripción
<code>id</code>	str	Identificador único del vértice
<code>adjacencies</code>	list[Edge]	Lista de aristas salientes
<code>add_adjacency(edge)</code>	método	Agrega una arista saliente

Edge — backend/models/edge.py

Representa una ruta directa entre dos aeropuertos. Contiene toda la metadata del vuelo.

Atributo	Tipo	Descripción
<code>origin</code>	str	Código IATA del aeropuerto de origen
<code>destination</code>	Airport	Referencia al objeto aeropuerto destino
<code>distanceKm</code>	float	Distancia de la ruta en kilómetros
<code>aircrafts</code>	list[str]	Tipos de aeronave disponibles para esa ruta
<code>routeSubsidized</code>	bool	Si es verdadero, el vuelo tiene costo cero (ruta subsidiada)
<code>flightTime</code>	float	Tiempo de vuelo en minutos (0 = calcular desde distancia)
<code>baseCost</code>	float	Costo base en USD (0 = calcular desde distancia + aeronave)
<code>minimumStay</code>	int	Estancia mínima en destino (minutos)
<code>is_blocked</code>	bool	Indica si la ruta está temporalmente bloqueada
<code>get_Weight()</code>	float	Retorna el peso primario (distancia en km)
<code>block()</code> / <code>unblock()</code>	método	Activa o desactiva el bloqueo de la ruta
<code>to_dict()</code> / <code>from_dict(data)</code>	método	Serialización / deserialización JSON

Airport — `backend/models/Airport.py`

Extiende `Vertex`. Representa un aeropuerto con toda su información turística y económica.

Atributo	Tipo	Descripción
<code>id</code>	str	Código IATA (ej. BOG, MDE, LIM)
<code>name</code>	str	Nombre completo del aeropuerto
<code>city</code>	str	Ciudad donde se ubica
<code>country</code>	str	País
<code>timeZone</code>	str	Zona horaria (ej. GMT-5)
<code>isHub</code>	bool	Aeropuerto hub (principal) o secundario
<code>accommodationCost</code>	float	Costo de alojamiento por noche en USD
<code>alimentationCost</code>	float	Costo de alimentación diaria en USD
<code>activities</code>	list[Activity]	Actividades turísticas disponibles
<code>jobs</code>	list[Job]	Trabajos disponibles para el viajero

Activity — `backend/models/Activity.py`

Actividad turística que el viajero puede realizar en un aeropuerto.

Atributo	Tipo	Descripción
<code>name</code>	str	Nombre de la actividad
<code>activityType</code>	str	Categoría: turismo, gastronomía, aventura, ocio
<code>durationMin</code>	int	Duración en minutos
<code>usdCost</code>	float	Costo en USD

Job — `backend/models/Job.py`

Trabajo disponible en un aeropuerto para que el viajero gane dinero.

Atributo	Tipo	Descripción
<code>name</code>	str	Nombre del trabajo
<code>hourlyRate</code>	float	Tarifa por hora en USD
<code>maxHours</code>	int	Horas máximas trabajables

Graph — `backend/models/graph.py`

Contenedor principal del grafo. Almacena todos los vértices (aeropuertos) e implementa todos los algoritmos de recorrido y optimización.

Atributo / Método	Descripción
<code>vertices</code> (dict)	Diccionario <code>{iata_code: Airport}</code> con todos los nodos
<code>config</code> (dict)	Configuración de aeronaves e intervalos del sistema
<code>add_vertex(airport)</code>	Agrega un aeropuerto al grafo
<code>get_vertex(id)</code>	Retorna un aeropuerto por código IATA
<i>Algoritmos — ver sección 3.3</i>	

3.3 Algoritmos implementados

Todos los algoritmos están definidos como funciones dentro de `backend/models/graph.py`. Operan sobre el grafo en memoria y no tienen efectos secundarios sobre los datos almacenados.

Algoritmo	Función	Propósito	Complejidad
Dijkstra simple	<code>dijkstra_simple(graph, start_id, dest_id)</code>	Ruta más corta por distancia. Omite aristas bloqueadas (<code>is_blocked=True</code>). Se usa para re-enrutamiento de emergencia.	$O((V+E) \log V)$
Dijkstra multi-criterio	<code>dijkstra_multi_criteria(graph, start, dest, weight_fn, allowed_vertices)</code>	Ruta óptima con función de peso inyectable. Permite optimizar por distancia, tiempo, costo o combinaciones lineales normalizadas. Soporta filtrado por vértices permitidos (solo hubs).	$O((V+E) \log V)$
BFS por peso	<code>breadthFirstSearch_traversal(graph, initial_vertex_id)</code>	Recorrido por niveles ordenando vecinos por peso de arista en cada nivel.	$O(V+E)$
DFS por peso	<code>depth_traversal(graph, initial_vertex_id)</code>	Recorrido en profundidad recursivo ordenando vecinos por peso.	$O(V+E)$
Bellman-Ford	<code>bellmanFord(graph, id_vInitial)</code>	Camino mínimo desde un origen. Soporta pesos negativos (no usados en el dominio aéreo, pero implementado con fines académicos).	$O(V \cdot E)$
Prim — MST	<code>prim_traversal(graph, initial_vertex_id)</code>	Árbol de expansión mínima. Convierte el dígrafo a grafo no dirigido antes de ejecutar. Propósito académico y de análisis de red.	$O(E^2)$
Ford-Fulkerson	<code>ford_fulkerson(graph, source_id, sink_id)</code>	Flujo máximo entre dos nodos. Mantiene grafo residual con aristas directas e inversas. Usa DFS para encontrar caminos aumentantes.	$O(V \cdot E^2)$
DFS de itinerarios	<code>find_itineraries_dfs(graph, start, budget, time_limit, transports, constraint, aircraft_config, overhead)</code>	Explora todos los itinerarios factibles desde un origen mediante DFS. En cada nodo calcula costos acumulados (vuelo, alojamiento, alimentación, actividades) y filtra según la restricción activa (presupuesto, tiempo, o ambas). Retorna lista de caminos con desglose completo.	$O(V^V)$ — exponencial

Cálculo de costos por vuelo

Los costos se calculan dinámicamente en función de la aeronave y la distancia:

```
# Tiempo de vuelo (minutos)
tiempo = distanciaKm * tiempoKm_por_aeronave + overheadVueloMin

# Costo de vuelo (USD)
costo = distanciaKm * costoKm_por_aeronave    (si la ruta NO está subsidiada)
costo = 0                                     (si la ruta está subsidiada)

# Configuración de aeronaves (valores del JSON)
"Avión Comercial": costoKm=0.18, tiempoKm=0.07
"Avión Regional":  costoKm=0.25, tiempoKm=0.15
"Hélice":          costoKm=0.12, tiempoKm=0.40
```

3.4 Capa de servicios

Los servicios encapsulan la lógica de negocio. Los controladores delegan en ellos; cada servicio mantiene una referencia al grafo cargado en memoria.

GraphLoadService — `backend/services/graphLoadService.py`

Parsea el archivo JSON y construye el grafo en memoria.

Método	Descripción
<code>load_from_json(json_data)</code>	Recibe el diccionario parseado del JSON. Crea objetos <code>Airport</code> y los conecta con objetos <code>Edge</code> según las aristas. Almacena la configuración de aeronaves. Retorna el objeto <code>Graph</code> construido.
<code>get_frontend_network_data()</code>	Transforma el grafo interno al formato esperado por el frontend: listas de <code>nodes</code> y <code>links</code> con todos sus atributos, más el objeto <code>config</code> .

RoutePerformanceService — `backend/services/routePerformanceService.py`

Optimización de rutas y generación de itinerarios automáticos.

Método	Descripción
<code>calculate_optimized_routes(origin, dest, criteria, exclude_secondary, preferred_transports)</code>	Para cada criterio solicitado, construye una función de peso personalizada y llama a <code>dijkstra_multi_criterio</code> . En criterios combinados (ej. distancia+tiempo), normaliza cada métrica respecto al máximo del grafo y promedia. Retorna un diccionario con la ruta y métricas para cada criterio.
<code>generate_automatic_itineraries(origin, budget, time_hours, preferred_transports)</code>	Llama a <code>find_itineraries_dfs</code> dos veces: una con restricción de presupuesto (Alternativa A — máximo destinos dentro del presupuesto) y otra con restricción de tiempo (Alternativa B — máximo destinos dentro del tiempo disponible). Selecciona el mejor camino por: más destinos → mayor diversidad de transporte → menor costo/tiempo.

InterruptionService — `backend/services/interruptionService.py`

Manejo de bloqueos y re-enrutamiento de emergencia.

Método	Descripción
<code>block_route(origin, dest, reason)</code>	Busca la arista entre origen y destino, activa <code>is_blocked=True</code> en ambas direcciones, y la registra en la lista <code>blocked_routes</code> .
<code>unblock_route(origin, dest)</code>	Localiza la ruta bloqueada y restaura <code>is_blocked=False</code> .
<code>clear_all_blocks()</code>	Desbloquea todas las rutas de una vez.
<code>get_network_status()</code>	Retorna: total de rutas, rutas activas, rutas bloqueadas, porcentaje bloqueado y lista detallada de rutas bloqueadas.
<code>recalculate_route(origin, dest)</code>	Llama a <code>dijkstra_simple</code> (que omite aristas bloqueadas) y retorna la ruta alternativa encontrada.
<code>handle_in_transit_interruption(segment_origin, final_dest)</code>	El avión en tránsito regresa al aeropuerto de inicio del segmento, y desde ahí se recalcula la ruta al destino final.

DynamicTravelService — `backend/services/dynamicTravelService.py`

Gestión del estado del viajero en tiempo real durante el viaje dinámico.

Método	Descripción
<code>get_next_step_options(state)</code>	Recibe el estado actual (nodo, presupuesto, tiempo transcurrido). Calcula: trabajos disponibles (si presupuesto < 35% del inicial), actividades opcionales, y todos los vuelos disponibles desde el nodo actual con sus costos totales (vuelo + alojamiento si aplica + alimentación si aplica). Retorna también una sugerencia heurística del mejor próximo movimiento.
<code>suggest_best_next_move(options)</code>	Heurística simple: elige el destino no visitado con menor costo total de segmento. Si todos están visitados, elige el de menor costo global.
<code>process_user_action(data)</code>	Procesa una acción del usuario (vuelo, trabajo, actividad). Actualiza el estado: descuenta/suma presupuesto, incrementa contadores de tiempo, registra en el historial. Retorna el nuevo estado completo.

ReportService / FinalReport — `backend/services/reportService.py`

Genera y serializa el reporte final del viaje.

Atributo	Descripción
<code>destinationsVisited</code>	Aeropuertos visitados con costos detallados
<code>flightsCovered</code>	Segmentos de vuelo realizados
<code>activitiesPerformed</code>	Actividades realizadas por destino
<code>workCompleted</code>	Trabajos realizados y ganancias
<code>totals</code>	Resumen: presupuesto inicial, gastado, ganado, saldo final, tiempo total

3.5 Controladores y API REST

Los controladores son blueprints de Flask. Reciben la petición HTTP, validan los parámetros, delegan en el servicio correspondiente y retornan la respuesta JSON.

Blueprint: `/api/graph` — `controllers/graphLoadController.py`

Método	Ruta	Payload (JSON)	Respuesta exitosa
POST	<code>/api/graph/load</code>	Objeto completo del archivo JSON con claves <code>nodos</code> , <code>aristas</code> , <code>configuracion</code>	<code>{ nodes, links, config }</code> listos para el renderizador
GET	<code>/api/graph/airport/<iata></code>	—	Objeto <code>Airport</code> completo con actividades y trabajos. HTTP 404 si no existe.

Blueprint: `/api/routes` — `controllers/routePerformanceController.py`

Método	Ruta	Payload (JSON)	Respuesta exitosa
POST	<code>/api/routes/optimize</code>	<code>{ origin, destination, criteria[], excludeSecondary, preferredTransports[] }</code>	Diccionario con una entrada por criterio: <code>{ "distance": { path[], total_metric, breakdown }, ... }</code>
POST	<code>/api/routes/itinerary</code>	<code>{ origin, budget, availableTime, preferredTransports[] }</code>	<code>{ alternative_a: { sequence[], segments[], total_cost_usd, ... }, alternative_b: { ..., total_time_hours } }</code>

Blueprint: `/api/dynamic` — `controllers/dynamicTravelController.py`

Método	Ruta	Payload (JSON)	Respuesta exitosa
POST	<code>/api/dynamic/next-options</code>	<code>{ current_node, initial_budget, current_budget, time_since_sleep, time_since_food, visited[] }</code>	<code>{ options: { jobs[], activities[], flights[] }, suggestion }</code>
POST	<code>/api/dynamic/process-action</code>	<code>{ action_type, action_details, current_state }</code>	<code>{ new_state: { current_node, current_budget, time_since_sleep, time_since_food, visited[], history[] } }</code>

Blueprint: `/api/interruptions` — `controllers/interruptionController.py`

Método	Ruta	Payload (JSON)	Descripción
POST	<code>/api/interruptions/block</code>	<code>{ origin, destination, reason }</code>	Bloquea la ruta en ambas direcciones
POST	<code>/api/interruptions/unblock</code>	<code>{ origin, destination }</code>	Desbloquea una ruta específica
GET	<code>/api/interruptions/status</code>	—	Estado de la red: total, activas, bloqueadas con porcentaje
POST	<code>/api/interruptions/recalculate</code>	<code>{ origin, destination }</code>	Calcula ruta alternativa evitando rutas bloqueadas
POST	<code>/api/interruptions/transit</code>	<code>{ origin, finalDestination }</code>	Regresa el avión al inicio del segmento y recalcula
POST	<code>/api/interruptions/clear</code>	—	Desbloquea todas las rutas del grafo

Blueprint: `/api/report` — `controllers/reportController.py`

Método	Ruta	Payload (JSON)	Respuesta exitosa
POST	<code>/api/report/generate</code>	Objeto <code>FinalReport</code> con destinos, vuelos, actividades, trabajos y totales	Mismo objeto validado y formateado

3.6 Formato del archivo de datos JSON

El sistema se inicializa cargando el archivo `backend/data/archivo_principal.json`. Su estructura es la siguiente:

```

{
  "nodos": [
    {
      "id": "BOG", // Código IATA (obligatorio, único)
      "nombre": "Aeropuerto Internacional El Dorado",
      "ciudad": "Bogotá",
      "pais": "Colombia",
      "zonaHoraria": "GMT-5",
      "esHub": true, // true = hub, false = aeropuerto secundario
      "costoAlojamiento": 50, // USD por noche
      "costoAlimentacion": 15, // USD por día
      "actividades": [
        {
          "nombre": "Tour por la ciudad",
          "tipo": "turismo", // turismo | gastronomía | aventura | ocio
          "duracionMin": 180, // duración en minutos
          "costoUSD": 25
        }
      ],
      "trabajos": [
        {
          "nombre": "Carga de equipaje",
          "tarifaHora": 8, // USD por hora
          "maxHoras": 6
        }
      ]
    }
  ],

  "aristas": [
    {
      "origen": "BOG", // código IATA del origen
      "destino": "MDE", // código IATA del destino
      "distanciaKm": 250,
      "aeronaves": ["Avión Comercial", "Avión Regional"],
      "rutaSubsidiada": false, // si true, costo del vuelo = 0
      "tiempoVuelo": 0, // 0 = calcular desde distancia
      "costoBase": 0, // 0 = calcular desde distancia y aeronave
      "estanciaMinima": 1440 // minutos (1440 = 1 día)
    }
  ],

  "configuracion": {
    "aeronaves": {
      "Avión Comercial": { "costoKm": 0.18, "tiempoKm": 0.07 },
      "Avión Regional": { "costoKm": 0.25, "tiempoKm": 0.15 },
      "Hélice": { "costoKm": 0.12, "tiempoKm": 0.40 }
    },
    "presupuestoMinimoPorc": 35, // % del presupuesto inicial bajo el cual aparecen trabajos
    "intervaloAlojamiento": 20, // horas sin dormir antes de cobrar alojamiento
    "intervaloAlimentacion": 8, // horas sin comer antes de cobrar alimentación
    "overheadVueloMin": 20 // minutos fijos de overhead por vuelo
  }
}

```

Las aristas son dirigidas. Si una ruta es bidireccional, debe aparecer dos veces en el arreglo `"aristas"`: una vez con origen A→B y otra con origen B→A.

3.7 Utilidades — `backend/utils/utils.py`

Función	Descripción
<code>readJson(route)</code>	Lee y parsea un archivo JSON desde la ruta indicada. Retorna el objeto Python resultante.
<code>writeJson(jsonData, route)</code>	Sobreescribe un archivo JSON con el contenido proporcionado.
<code>addContentToJson(newContent, route)</code>	Agrega contenido a un archivo JSON existente, añadiendo el separador de coma necesario.

4. Frontend

4.1 Estructura de archivos

```

frontend/
├─ views/
│   ├─ index.html          # Landing page
│   └─ app.html            # Aplicación principal (SPA)
├─ css/
│   ├─ main.css            # Estilos de la landing page
│   └─ app.css             # Estilos de la aplicación principal
├─ js/
│   ├─ pages/
│   │   ├─ main.js         # Punto de entrada - inicialización y atajos globales
│   │   ├─ graphLoadManager.js # R1: carga y visualización del grafo
│   │   ├─ routePerformanceManager.js # R2: planificación de rutas
│   │   ├─ dynamicTravelManager.js # R3: viaje dinámico paso a paso
│   │   ├─ interruptionManager.js # R4: gestión de interrupciones
│   │   └─ reportManager.js # R5: reporte final
│   └─ models/
│       └─ graphRenderer.js # Motor de renderizado Canvas
│   └─ services/
│       ├─ graphLoadService.js # Cliente API /api/graph
│       ├─ routePerformanceService.js # Cliente API /api/routes
│       ├─ dynamicTravelService.js # Cliente API /api/dynamic
│       ├─ interruptionService.js # Cliente API /api/interruptions
│       └─ reportService.js # Cliente API /api/report
├─ utils/
│   └─ utils.js            # Utilidades compartidas (toast, modales, fetch)
└─ assets/
    └─ images/
        ├─ cielo.jpg
        ├─ logo.png
        └─ ucaldas.png

```

4.2 Vistas HTML

`frontend/views/index.html` — *Landing Page*

Página de bienvenida con fondo de cielo, logo de la institución y un botón "Ingresar al Sistema" que redirige a `app.html`. No contiene lógica JavaScript; es puramente presentacional.

`frontend/views/app.html` — *Aplicación Principal*

Contiene toda la estructura DOM de la aplicación en una sola página (SPA). Las secciones principales son:

- **Header:** Título "SkyRoute Planner", botones de carga JSON y reinicio.

- **Toolbar:** Botones de módulos (Planificación Básica, Planificación Avanzada, Interrumpir Ruta, Exportar Reporte) y estadísticas de la red.
- **Panel lateral izquierdo (350 px fijo):** Cuatro paneles que se muestran según el contexto:
 - *Información del Aeropuerto* — visible al seleccionar un nodo en el canvas.
 - *Itinerario Calculado* — visible después de una planificación de ruta.
 - *Resumen del Viaje* — visible durante el viaje dinámico.
 - *Reporte Final* — visible al finalizar el viaje.
- **Canvas (área central):** Renderizado del grafo. Incluye leyenda y controles de zoom.
- **Modales:** Diez modales para las distintas interacciones del usuario.

4.3 Flujo de la interfaz de usuario

Módulo R1 — Carga y visualización de la red

1. El usuario hace clic en "Cargar Red Aérea (JSON)".
2. Selecciona el archivo `archivo_principal.json` desde el explorador de archivos.
3. El frontend lee el archivo localmente y envía el JSON al endpoint `POST /api/graph/load`.
4. El backend construye el grafo en memoria y retorna los datos formateados para el renderizador.
5. El `GraphRenderer` ejecuta la simulación de fuerzas y dibuja el grafo en el canvas.
6. Al hacer clic en un nodo, el panel lateral muestra la información completa del aeropuerto.

Módulo R2 — Planificación de rutas

El usuario accede mediante el botón "Planificación Básica". Hay dos modos:

- **Planificación Básica (itinerario automático):** El usuario ingresa origen, presupuesto máximo y tiempo disponible. El sistema genera dos alternativas: la A maximiza destinos dentro del presupuesto, y la B maximiza destinos dentro del tiempo.
- **Mejor Ruta:** El usuario ingresa origen, destino, criterios de optimización (distancia, tiempo, costo, o combinaciones), tipos de aeronave preferidos y si excluir aeropuertos secundarios. El sistema retorna una ruta óptima por cada criterio seleccionado.

Para cada resultado aparecen dos botones: "Visualizar" (resalta la ruta estáticamente en el canvas) y "Ejecutar Ruta" (anima el vuelo con un punto de color sobre el canvas).

Módulo R3 — Viaje dinámico paso a paso

1. El usuario abre "Planificación Avanzada", elige origen y presupuesto inicial.
2. El sistema consulta `/api/dynamic/next-options` con el estado actual.
3. El usuario puede: trabajar (si el presupuesto está bajo el 35%), realizar actividades, o elegir un vuelo.
4. Al confirmar un vuelo, el canvas anima el desplazamiento del avión hasta el destino.
5. El panel lateral muestra en tiempo real: ubicación, presupuesto, tiempo transcurrido, contadores de alojamiento/alimentación e historial de acciones.
6. El usuario puede terminar el viaje en cualquier momento para generar el reporte.

Módulo R4 — Gestión de interrupciones

1. El usuario hace clic en "Interrumpir Ruta" y selecciona una ruta activa y un motivo.
2. El backend bloquea la arista en ambas direcciones.

- 3. El canvas actualiza la arista afectada a color rojo con línea discontinua.
- 4. Si el avión está volando actualmente por ese segmento (en tránsito): el canvas anima el regreso al aeropuerto de origen, se dispara el evento `skyroute:flightInterrupted`, y el `dynamicTravelManager` revierte el estado al snapshot previo al vuelo.
- 5. El usuario puede gestionar todos los bloqueos, desbloquear rutas y solicitar recálculo de alternativas.

Módulo R5 — Reporte final

- 1. Se activa automáticamente al terminar el viaje dinámico, o manualmente con el botón "Exportar Reporte".
- 2. Resalta el camino completo del viaje sobre el canvas.
- 3. Muestra el reporte en un modal con KPIs (presupuesto inicial, gastado, ganado, saldo final, tiempo total, número de destinos, vuelos y actividades) y tablas detalladas de cada categoría.

4.4 Servicios JavaScript — clientes API

Los servicios son módulos ES6 que encapsulan las llamadas HTTP a la API del backend. Usan la función `_apiFetch` de `utils/utils.js` como capa base. La URL base es `http://localhost:5000/api`.

Archivo	Funciones exportadas	Endpoints consumidos
<code>graphLoadService.js</code>	<code>loadGraphFromJSON(jsonData)</code> <code>getAirportDetails(iataCode)</code>	<code>POST /api/graph/load</code> <code>GET /api/graph/airport/{iata}</code>
<code>routePerformanceService.js</code>	<code>calculateOptimizedRoutes(origin, dest, criteria, excludeSecondary, transports)</code> <code>generateItineraries(origin, budget, time, transports)</code>	<code>POST /api/routes/optimize</code> <code>POST /api/routes/itinerary</code>
<code>dynamicTravelService.js</code>	<code>getNextStepOptions(state)</code> <code>processTravelAction(payload)</code>	<code>POST /api/dynamic/next-options</code> <code>POST /api/dynamic/process-action</code>
<code>interruptionService.js</code>	<code>blockRoute(o, d, reason)</code> <code>unblockRoute(o, d)</code> <code>getNetworkStatus()</code> <code>recalculateRoute(o, d)</code> <code>handleTransitInterruption(segOrigin, finalDest)</code> <code>clearAllBlocks()</code>	<code>POST /api/interruptions/block</code> <code>POST /api/interruptions/unblock</code> <code>GET /api/interruptions/status</code> <code>POST /api/interruptions/recalculate</code> <code>POST /api/interruptions/transit</code> <code>POST /api/interruptions/clear</code>
<code>reportService.js</code>	<code>generateReport(payload)</code>	<code>POST /api/report/generate</code>

4.5 Page Managers

Cada Page Manager es un módulo JS responsable de un módulo funcional completo. Se inicializan en el evento `DOMContentLoaded` y se comunican entre sí exclusivamente mediante eventos personalizados del DOM.

graphLoadManager — `js/pages/graphLoadManager.js`

Responsable del módulo R1. Carga el JSON, inicializa el `GraphRenderer`, maneja los clics sobre nodos, actualiza el panel de información del aeropuerto y los contadores de la toolbar.

Exporta globalmente: `getRenderer()`, `getNetworkData()`, `requireNetwork(fn)`.

routePerformanceManager — `js/pages/routePerformanceManager.js`

Responsable del módulo R2. Gestiona los dos modales de planificación (básica y mejor ruta), llama a los servicios correspondientes y renderiza los resultados en el panel lateral con botones de visualización y ejecución de animación.

dynamicTravelManager — `js/pages/dynamicTravelManager.js`

Responsable del módulo R3. Es el manager más complejo: mantiene el estado del viaje (`_travelState`), orquesta la secuencia de modales (trabajos → actividades → transporte), gestiona la animación de vuelo y escucha eventos de interrupción para revertir el estado cuando un vuelo es bloqueado en tránsito.

Estado guardado en `window._skyRouteTravelState` y snapshot previo al vuelo en `window._skyRoutePreFlightSnapshot`.

interruptionManager — `js/pages/interruptionManager.js`

Responsable del módulo R4. Permite bloquear rutas activas, detectar si el avión está en tránsito por esa ruta (inspeccionando `renderer._flightAnim`), animar el regreso del avión, gestionar la lista de bloqueos y calcular rutas alternativas.

reportManager — `js/pages/reportManager.js`

Responsable del módulo R5. Se activa cuando el evento `skyroute:journeyEnded` es disparado. Construye el objeto reporte desde el estado final del viaje, lo envía al backend, resalta el camino completo en el canvas y muestra el modal de reporte con todos los detalles.

main.js — `js/pages/main.js`

Punto de entrada de la aplicación. Verifica que todos los módulos están cargados correctamente, registra los atajos de teclado globales y expone la configuración compartida en `window._skyRouteConfig`.

Atajo	Acción
ESC	Cerrar todos los modales abiertos
Ctrl + o / Ctrl + Home	Resetear zoom y centrar el grafo
Ctrl + +	Acercar (zoom in)
Ctrl + -	Alejar (zoom out)

4.6 Motor de renderizado — GraphRenderer

Clase definida en `js/models/graphRenderer.js`. Es responsable de toda la representación visual del grafo en el elemento `<canvas>` de HTML5.

Simulación de fuerzas

El grafo usa un layout de simulación de fuerzas (force-directed). Al cargar el grafo, los nodos se posicionan inicialmente de forma aleatoria y la simulación aplica tres fuerzas iterativamente hasta estabilizarse (~500 iteraciones a 60 fps):

- **Repulsión de Coulomb:** los nodos se repelen entre sí proporcionalmente al inverso del cuadrado de la distancia.
- **Atracción de Hooke:** las aristas actúan como resortes que atraen los nodos conectados hacia una longitud de reposo.
- **Gravedad central:** atrae suavemente todos los nodos hacia el centro del canvas para evitar que se dispersen.

Pipeline de dibujo

En cada frame, el canvas se redibuja completamente en este orden:

1. Limpiar canvas y aplicar transformación de vista (pan + zoom).
2. Dibujar la cuadrícula de fondo (`_drawGrid`).
3. Dibujar todas las aristas (`_drawAllEdges`).
4. Dibujar todos los nodos (`_drawAllNodes`).
5. Dibujar el indicador de vuelo animado (`_drawFlightBall`), si existe.

Esquema de colores

Elemento	Color	Condición
Nodo hub	Rojo	<code>isHub = true</code>
Nodo secundario	Azul	<code>isHub = false</code>
Nodo seleccionado	Verde	Clic del usuario
Nodo actual del viajero	Cian	<code>currentNode</code> del viaje dinámico
Arista normal	Gris slate	Estado por defecto
Arista bloqueada	Rojo / línea discontinua	<code>is_blocked = true</code>
Arista resaltada	Verde	Parte de la ruta calculada
Punto de aeronave comercial	Morado	Indicador bajo el nodo
Punto de aeronave regional	Cian	Indicador bajo el nodo
Punto de hélice	Ámbar	Indicador bajo el nodo

Animación de vuelo

El método `animateFlight(path, preferredTransports, onComplete)` anima un punto de color viajando por los segmentos del camino indicado. La duración es de 25 segundos por segmento. El estado de la animación se almacena en `renderer._flightAnim`, que el `interruptionManager` inspecciona para detectar vuelos en tránsito.

El método `animateReturn(from, to, startT, onComplete)` anima el regreso del avión cuando una ruta es bloqueada en tránsito.

Interacción del usuario

Interacción	Resultado
Clic sobre un nodo	Selecciona el nodo, dispara <code>skyroute:nodeSelected</code>
Clic sobre una arista	Selecciona la arista, dispara <code>skyroute:edgeSelected</code>
Clic en espacio vacío	Deselecciona, dispara <code>skyroute:deselect</code>
Arrastrar (drag)	Desplaza el grafo (pan)
Rueda del ratón	Zoom (factor 1.12× por clic de rueda)
Ctrl+o	Resetea la vista al estado inicial

4.7 Sistema de eventos personalizados

Los eventos permiten desacoplar los módulos. Se despachan con `document.dispatchEvent(new CustomEvent('skyroute:nombre', { detail: {...} }))` y se escuchan con `document.addEventListener('skyroute:nombre', handler)`. La tabla completa se encuentra en la sección 2.3 de este manual.

4.8 Estilos CSS

El proyecto usa dos archivos CSS con variables personalizadas (custom properties). No hay ningún framework de CSS; todo el estilo es artesanal.

`css/main.css` — *Landing page*

Fondo de imagen de cielo, encabezado con logo, sección hero centrada y botón de ingreso.

`css/app.css` — *Aplicación principal*

Usa un layout de cuadrícula CSS con tres zonas: header (70 px), toolbar (60 px), y el área principal dividida en panel lateral (350 px fijo) + canvas (flexible).

Variables de color definidas en `:root`:

Variable	Propósito
<code>--primary-color</code>	Azul principal — botones y énfasis
<code>--danger-color</code>	Rojo — advertencias y rutas bloqueadas
<code>--success-color</code>	Verde — confirmaciones y rutas resaltadas
<code>--info-color</code>	Azul informativo — notificaciones
<code>--warning-color</code>	Naranja — alertas

Los modales usan una animación de fade-in sobre un backdrop semitransparente. Las notificaciones toast aparecen en la esquina inferior derecha y se auto-descartan.

5. Requisitos del Sistema

5.1 Backend

Componente	Versión mínima	Notas
Python	3.8	Se usan f-strings y type hints opcionales compatibles desde 3.8.
Flask	2.0	Se requiere la API de Blueprints y <code>render_template</code> .
Flask-CORS	3.0	Permite peticiones cross-origin desde el frontend.
pip	21.0	Para instalación de dependencias.
Sistema operativo	Windows 10 / Ubuntu 20.04 / macOS 11	Cualquier SO con soporte de Python 3.8+.
RAM disponible	512 MB mínimo	El grafo vive en memoria; redes pequeñas (<100 nodos) requieren muy poca memoria.

5.2 Frontend (navegadores soportados)

El frontend usa **ES6 Modules** (`type="module"`), **Canvas API** y **CustomEvent**. No requiere ningún bundler, transpilador ni dependencias externas.

Navegador	Versión mínima	Notas
Google Chrome	80 (lanzado Feb 2020)	Plataforma de desarrollo y prueba principal.
Mozilla Firefox	75 (lanzado Abr 2020)	Soporte nativo de ES6 Modules.
Microsoft Edge	80 (Chromium-based, Feb 2020)	Misma base que Chrome, total compatibilidad.
Safari	14 (lanzado Sep 2020)	ES6 Modules soportados desde Safari 14.
Opera	67 (lanzado Mar 2020)	Basado en Chromium.
Internet Explorer	No soportado. IE no implementa ES6 Modules ni CustomEvent constructor estándar.	

Se recomienda usar la versión más reciente disponible de cualquier navegador soportado. El rendimiento del canvas mejora notablemente con hardware acceleration activa (habilitada por defecto en todos los navegadores modernos).

5.3 Red

Requisito	Detalle
Puerto 5000 disponible	Flask escucha en <code>0.0.0.0:5000</code> . Verificar que no haya otro proceso ocupando ese puerto.
Conexión localhost	El frontend hace fetch a <code>http://localhost:5000/api</code> . No se requiere internet.
Firewall	Si se accede desde otra máquina en la misma red, asegurarse de que el firewall permita el puerto 5000.

6. Instalación y Ejecución

6.1 Instalación de dependencias

```
# Verificar versión de Python (debe ser 3.8 o superior)
python --version

# Instalar las dependencias del backend
pip install flask flask-cors

# Verificar instalación
pip show flask
pip show flask-cors
```

6.2 Ejecutar el servidor

```
# Desde la raíz del proyecto, entrar al directorio backend
cd backend

# Iniciar Flask
python app.py
```

El terminal mostrará:

```
* Running on http://0.0.0.0:5000
* Debug mode: on
```

6.3 Acceder a la aplicación

1. Abrir un navegador compatible (Chrome 80+, Firefox 75+, Edge 80+, Safari 14+).
2. Navegar a: `http://localhost:5000`
3. La aplicación mostrará directamente la interfaz principal (`app.html`).
4. Hacer clic en "Cargar Red Aérea (JSON)" y seleccionar el archivo `backend/data/archivo_principal.json`.
5. La red aérea se visualizará en el canvas. El sistema está listo para usar.

6.4 Reiniciar el sistema

Hacer clic en el botón "Reiniciar" de la interfaz limpia el grafo de la memoria sin necesidad de reiniciar el servidor. Para un reinicio completo del estado del servidor, detener el proceso (Ctrl+C) y volver a ejecutar `python app.py`.
